 # Task 1: Capablilities

From the CampusEats brief, the following distinct capabilities (jobs) are identified:

1. **Browse Menu** — students view available food items from a vendor

2. **Place Order** — students create a new order with selected items

3. **Make Payment** — students pay for their order online

4. **Update Order Status** — vendors mark orders as preparing/ready/completed

5. **Track Delivery** — students check live delivery status of their order

6. **Cancel Order** — students cancel an order before it is prepared

7. **Manage Vendor Menu** — vendors add/edit/remove menu items and prices

8. **Manage Profile** — students manage their account info and delivery addresses

 # Task 3: Service Contracts

Each contract lists only the operations other services may call — inputs, outputs, and errors. No table names, internal IDs, or SQL are exposed; callers depend only on the promise, never the implementation.

Contract: Accounts Service

Owns: users, addresses, login

Operation	Input	Output	Errors
getAddress	studentId	address details	student not found
register	name, email, password	studentId, status	email already exists
login	email, password	studentId, token	invalid credentials
updateProfile	studentId, fields	status: updated	student not found

Contract: Catalogue Service

Owns: restaurants, menus, prices

Operation	Input	Output	Errors
listRestaurants	area / filter	restaurants[]	—
getMenu	restaurantId	menu items + prices	not found
checkItem	itemId	available?, price	not found

Contract: Orders Service (main service)

Owns: carts, orders, status

Operation	Input	Output	Errors
addToCart	studentId, itemId, qty	cart	item unavailable
placeOrder	studentId, cart, addressId, paymentMethodId	orderId, status, total	empty cart, bad address, item unavailable
getOrder	orderId	order + status	not found
cancelOrder	orderId	status: cancelled	too late to cancel

Contract: Payments Service (main service)

Owns: transactions

Operation	Input	Output	Errors
charge	orderId, amount, paymentMethodId	transactionId, status: success	payment declined, invalid method
refund	transactionId	status: refunded	transaction not found
getTransaction	transactionId	transaction details	not found

Contract: Delivery Service

Owns: riders, assignments

Operation	Input	Output	Errors
assignRider	orderId, addressId	riderId, estimatedMinutes	no rider available
trackDelivery	orderId	riderId, currentStatus, location	order not found

Contract: Notifications Service

Owns: message log

Operation	Input	Output	Errors
send	studentId, message, type	status: sent	delivery failed

##Task 4: Services, Contracts and Schema

Service: Orders

Inputs

- studentId — identifies the student placing the order
- items: [{itemId, qty}] — items selected from the cart, with quantities
- deliveryAddressId — reference to the student's saved delivery address
- paymentMethodId — reference to the student's chosen payment method

Outputs

- orderId — unique identifier for the newly created order
- status: 'placed' — confirms the order was created successfully
- total — total amount charged for the order
- estimatedMinutes — estimated delivery time

Error cases

- item unavailable — one or more cart items are no longer available (checked via Catalogue.checkItem)

- payment declined — payment could not be processed (returned by Payments.charge)
- invalid address — deliveryAddressId does not exist or is not valid
- empty cart — the request was made with no items in the cart

Hidden from callers

- The internal structure of the orders table and how order records are stored
- How orderId values are generated
- The logic used to select and assign a rider — handled internally by Delivery
- Which payment gateway is used — handled internally by Payments
- Any retry, queuing, or internal service-to-service communication

Orchestration — what happens internally

1. Orders calls Catalogue.checkItem to verify each item is still available and confirm its price
2. Orders calls Payments.charge to take payment for the total amount
3. Orders calls Delivery.assignRider to schedule a rider
4. Orders calls Notifications.send to confirm the order to the student
5. Orders returns orderId, status, total, and estimatedMinutes to the student

Orders owns nothing beyond the order itself. Every fact it needs — price, payment, rider — is obtained by calling another service's contract, never by reading that service's data directly.

task6:

CampusEats — Validation Table

Task 6, Assignment 2: Design CampusEats — Services, Contracts and Schema

Service	Reachable	Self-contained	Has a contract	Independent	Loosely coupled	Notes
Accounts	Yes	Yes	Yes	Yes	Yes	Passes all five properties
Catalogue	Yes	Yes	Yes	Yes	Yes	Passes all five properties

Service	Reachable	Self-contained	Has a contract	Independent	Loosely coupled	Notes
Orders	Yes	Yes	Yes	Yes	Yes	Orchestrates other services but owns only carts, orders, order_items
Payments	Yes	Yes	Yes	Yes	Yes	Passes all five properties
Delivery	Yes	Yes	Yes	Yes	Yes	Passes all five properties
Notifications	Yes	Yes	Yes	Yes	Yes	Passes all five properties

Reasoning per property

- Reachable over a network — every service exposes operations (checkItem, charge, assignRider) that other services call remotely, not through direct code calls or shared memory
- Self-contained — each service does its entire job on its own and owns its own data. Payments handles the full charge and refund flow without needing another service's internal logic
- Has a contract — every service has a documented set of operations with inputs, outputs, and errors, so any caller knows exactly what to send and expect back
- Independent — each service can change its internal implementation (Payments switching gateway, Catalogue rebuilding its database) without affecting callers, since callers only depend on the contract
- Loosely coupled — no service reads another service's table directly. Orders never queries Catalogue's menu_items table, it calls Catalogue.checkItem and trusts the response